

PROFESSIONAL MOCK INTERVIEW PLATFORM

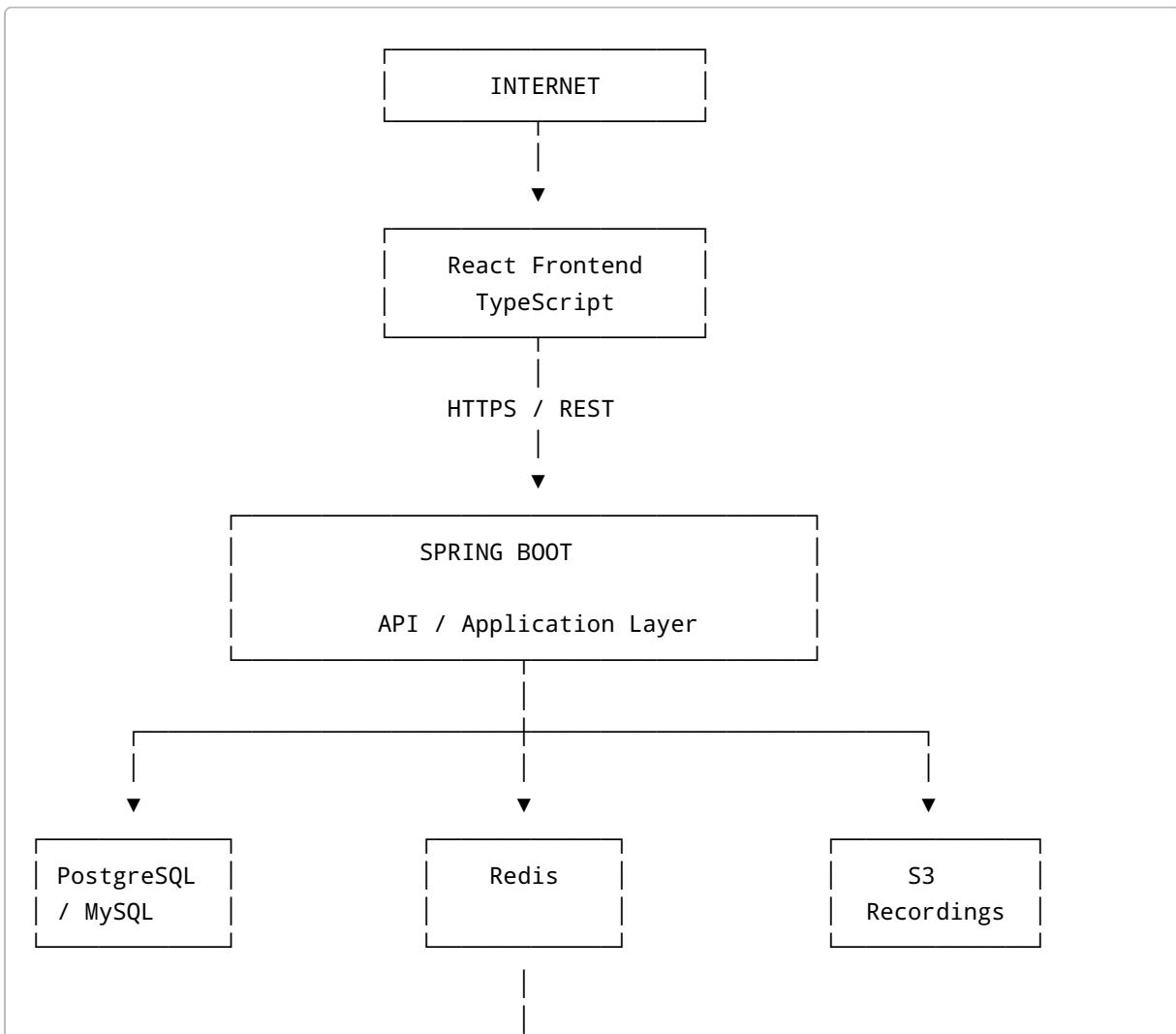
SYSTEM ARCHITECTURE DOCUMENT

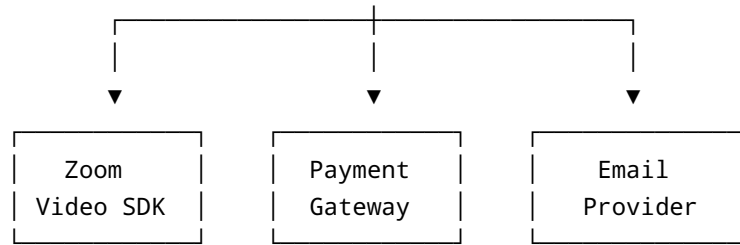
1. Architecture Overview

The platform will initially use a **Modular Monolith Architecture** using Spring Boot.

The architecture will be designed so that individual modules can later be extracted into microservices if the platform reaches significant scale.

The initial architecture:





2. Frontend Architecture

Frontend technology:

React
TypeScript
Vite
React Router
React Query
Tailwind CSS

The frontend contains three primary applications/views.

Candidate Portal
Interviewer Portal
Admin Portal

3. Candidate Frontend

Candidate pages:

Login
Register
Dashboard
Profile
Companies
Interviewers
Interview Search
Interviewer Details
Availability
Booking

- Payment
- Upcoming Interviews
- Interview Room
- Interview History
- Feedback
- Performance
- Notifications
- Settings

4. Interviewer Frontend

Interviewer pages:

- Login
- Registration
- Verification
- Dashboard
- Profile
- Availability
- Upcoming Interviews
- Interview Room
- Completed Interviews
- Feedback
- Earnings
- Payouts
- Notifications
- Settings

5. Admin Frontend

Admin pages:

- Login
- Dashboard
- Candidates
- Interviewers
- Verification Queue
- Companies
- Designations
- Domains

```
Pricing
Interviews
Payments
Refunds
Recordings
Feedback
Reports
Audit Logs
Settings
```

6. Backend Architecture

The backend will be implemented using Spring Boot.

Recommended modules:

```
Authentication
User Management
Candidate Management
Interviewer Management
Verification
Company Management
Designation Management
Domain Management
Scheduling
Booking
Payment
Interview
Video Session
Recording
Feedback
Rating
Notification
Admin
Audit
```

7. Spring Boot Package Structure

```
com.mockinterview
|
```

- └─ auth
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity
 - └─ dto
 - └─ security

- └─ user
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity
 - └─ dto

- └─ candidate
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity
 - └─ dto

- └─ interviewer
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity
 - └─ dto

- └─ verification
 - └─ controller
 - └─ service
 - └─ provider

- └─ company
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity

- └─ designation
 - └─ controller
 - └─ service
 - └─ repository
 - └─ entity

```
|— domain
|   |— controller
|   |— service
|   |— repository
|   └— entity
|
|— scheduling
|   |— controller
|   |— service
|   |— repository
|   └— entity
|
|— booking
|   |— controller
|   |— service
|   |— repository
|   └— entity
|
|— payment
|   |— controller
|   |— service
|   |— repository
|   |— webhook
|   └— provider
|
|— interview
|   |— controller
|   |— service
|   |— repository
|   └— entity
|
|— video
|   |— service
|   └— provider
|
|— recording
|   |— controller
|   |— service
|   └— provider
|
|— feedback
|   |— controller
|   |— service
|   |— repository
|   └— entity
|
```

```
├── rating
│   ├── controller
│   ├── service
│   └── entity
├── notification
│   ├── service
│   └── provider
├── admin
│   ├── controller
│   └── service
├── audit
│   ├── service
│   └── entity
└── common
    ├── exception
    ├── response
    ├── security
    ├── validation
    └── util
```

8. Database Architecture

Recommended database:

PostgreSQL

The following tables are recommended.

Authentication

```
users
roles
user_roles
refresh_tokens
```

Candidate

```
candidate_profiles  
candidate_skills  
candidate_experience
```

Interviewer

```
interviewer_profiles  
interviewer_domains  
interviewer_verifications  
interviewer_documents  
interviewer_ratings
```

Company

```
companies  
company_domains
```

Designation

```
designations  
designation_levels  
company_designations
```

Domain

```
interview_domains  
skills  
domain_skills
```

Scheduling

interviewer_availability
interview_slots

Booking

interviews
interview_participants

Payment

payments
payment_transactions
refunds

Feedback

feedback
feedback_categories
feedback_scores
feedback_comments

Recording

recordings

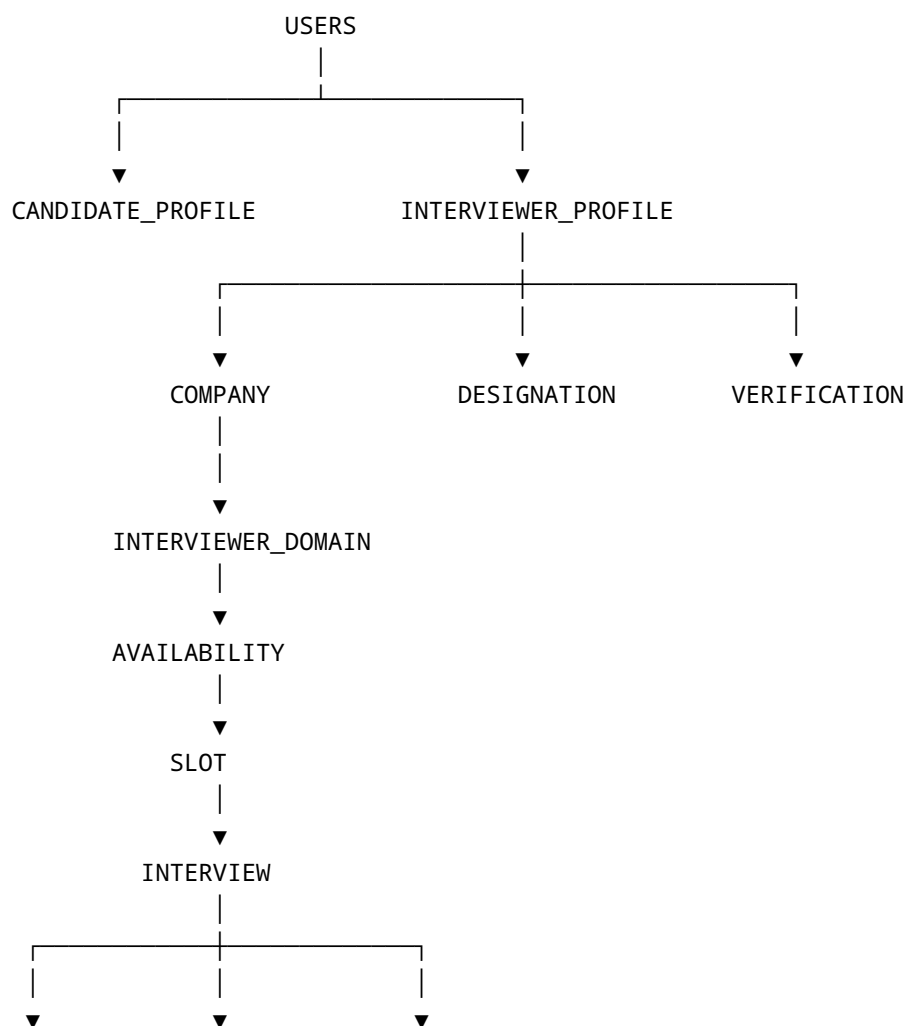
Notifications

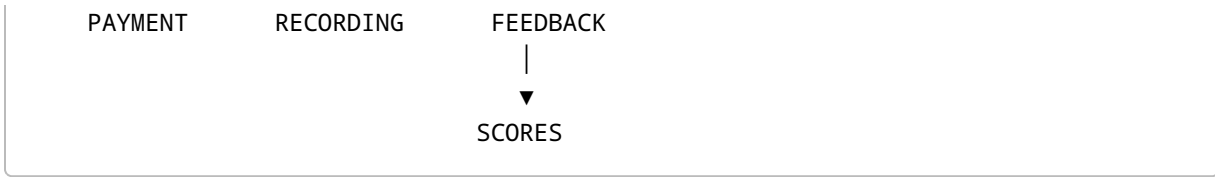
notifications
notification_preferences

Audit

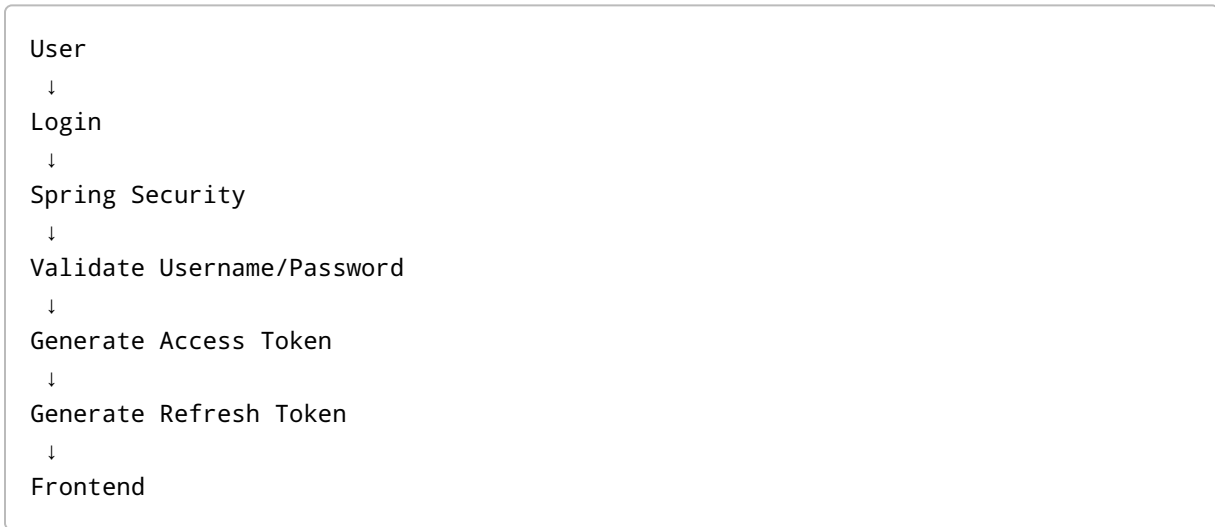
audit_logs

9. Entity Relationship Overview

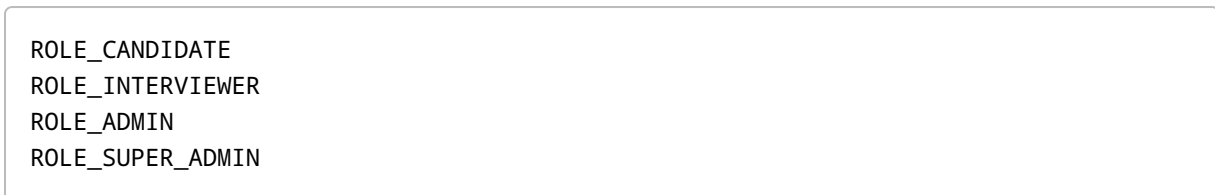




10. Authentication Architecture

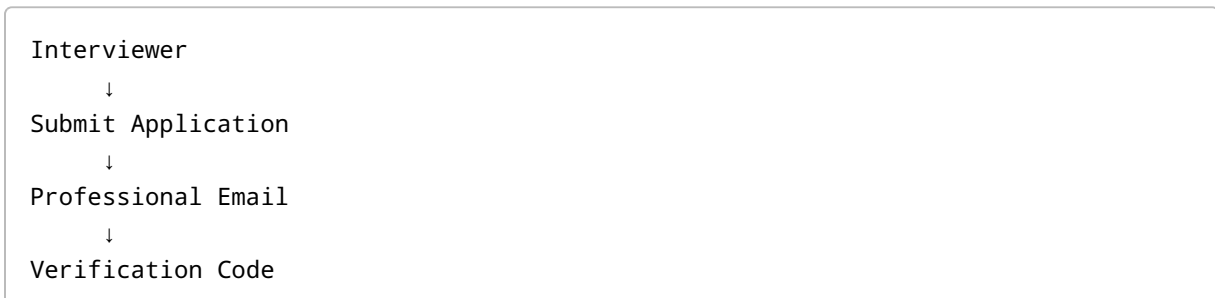


Roles:



All sensitive backend operations must use authorization checks.

11. Interviewer Verification Architecture



↓
Domain Verification
↓
Professional Verification
↓
Admin Review
↓
APPROVED
↓
Marketplace

Verification state machine:

PENDING
↓
EMAIL_VERIFIED
↓
UNDER_REVIEW
↓
APPROVED

Failure states:

REJECTED
SUSPENDED

12. Anonymous Identity Architecture

The system must separate:

PRIVATE IDENTITY

from:

PUBLIC MARKETPLACE PROFILE

Example:

PRIVATE

User ID:

984821

Name:

John Doe

Email:

john@company.com

Phone:

XXXXXXXXXX

LinkedIn:

private

Public:

Interviewer ID:

INT-PUB-4921

Company:

Cognizant

Designation:

Senior Software Engineer

Experience:

8+ Years

Domain:

Java / Spring Boot

Rating:

4.8

The candidate API must never return private identity fields.

13. Interview Search Architecture

Candidate sends:

```
Company
Domain
Candidate Level
Preferred Date
Duration
```

Backend:

```
Search Interviewers
  ↓
Filter Company
  ↓
Filter Domain
  ↓
Filter Verification
  ↓
Filter Interviewer Level
  ↓
Filter Availability
  ↓
Calculate Price
  ↓
Return Public Profiles
```

14. Eligibility Engine

Create a dedicated service:

```
InterviewEligibilityService
```

Responsibilities:

```
verifyCandidateLevel()
verifyInterviewerLevel()
verifyDomain()
verifyCompany()
verifyInterviewerStatus()
verifyAvailability()
```

Example:

```
Candidate Level = 2  
Interviewer Level = 4
```

```
ELIGIBLE
```

15. Pricing Engine

Create:

```
PricingService
```

Input:

```
Company  
Designation  
Domain  
Duration  
Interviewer Level
```

Output:

```
Final Interview Price  
Platform Commission  
Interviewer Earnings
```

Example:

```
Price = ₹1,500  
  
Platform Commission = ₹300  
  
Interviewer Earnings = ₹1,200
```

16. Scheduling Architecture

Interviewer creates availability:

Monday
18:00 - 21:00

Backend generates:

18:00
19:00
20:00

Each slot has a status:

AVAILABLE
HELD
BOOKED
BLOCKED
EXPIRED

17. Slot Locking

When candidate begins payment:

AVAILABLE
↓
HELD
↓
Payment
↓
BOOKED

If payment fails:

HELD
↓
AVAILABLE

A timeout should automatically release abandoned slots.

18. Payment Architecture

```
Candidate
  ↓
Spring Boot
  ↓
Create Payment Order
  ↓
Payment Provider
  ↓
Payment
  ↓
Provider Webhook
  ↓
Spring Boot
  ↓
Verify Signature
  ↓
SUCCESS
  ↓
Confirm Booking
```

The payment webhook is the source of truth.

19. Interview Architecture

```
BOOKING CONFIRMED
  ↓
INTERVIEW CREATED
  ↓
VIDEO SESSION CREATED
  ↓
SESSION TOKEN GENERATED
  ↓
CANDIDATE + INTERVIEWER JOIN
  ↓
INTERVIEW
  ↓
SESSION ENDS
  ↓
RECORDING PROCESSED
```

↓
FEEDBACK

20. Video Provider Abstraction

The backend should not directly depend on Zoom everywhere.

Create:

VideoProvider

with methods conceptually such as:

```
createSession()  
generateParticipantToken()  
startRecording()  
stopRecording()  
getRecording()  
endSession()
```

Implementation:

```
VideoProvider  
├── ZoomVideoProvider  
└── FutureWebRTCProvider
```

This prevents vendor lock-in.

21. Recording Architecture

```
Video Session  
↓  
Cloud Recording  
↓  
Recording Completed Event  
↓
```

```
Backend Webhook
  ↓
Recording Metadata
  ↓
Private S3
```

Recording metadata:

```
recording_id
interview_id
provider
provider_recording_id
storage_key
duration
file_size
status
created_at
expires_at
```

22. Recording Security

Candidate:

```
GET /api/v1/interviews/{id}/recording
```

Backend checks:

```
Is authenticated?
  ↓
Is candidate part of interview?
  ↓
Is recording available?
  ↓
Is candidate allowed access?
  ↓
Generate signed URL
```

The storage bucket must remain private.

23. Feedback Architecture

```
Interview Completed
  ↓
Feedback Pending
  ↓
Interviewer Opens Feedback Form
  ↓
Scores Categories
  ↓
Writes Comments
  ↓
Submit
  ↓
Feedback Published
  ↓
Candidate Notification
```

24. Feedback Categories

Generic:

```
Communication
Problem Solving
Technical Knowledge
Confidence
Coding
```

Domain-specific:

```
Java
Spring Boot
SQL
Microservices
System Design
```

Different domains should be able to have different feedback templates.

25. Candidate Analytics

Backend calculates:

```
Average Score
Domain Score
Category Score
Progress
Weaknesses
Strengths
```

Example:

```
Java
Previous: 5.5
Current: 8.0
Improvement: +2.5
```

26. Notification Architecture

Use asynchronous notification processing where possible.

```
Spring Boot
  ↓
Notification Event
  ↓
Queue / Redis
  ↓
Notification Worker
  ↓
Email Provider
```

Notifications can include:

```
Booking Confirmation
Interview Reminder
Payment Confirmation
Feedback Available
Cancellation
```

Refund
Interviewer Approval

27. Caching

Redis can be used for:

```
Available slots
Company list
Domain list
Designation list
Pricing configuration
Session data
Rate limiting
Temporary slot locks
```

Do not cache sensitive information unnecessarily.

28. Audit Logging

Every sensitive operation should be logged.

Examples:

```
Interviewer Approved
Interviewer Rejected
Payment Created
Payment Confirmed
Refund Created
Recording Accessed
Feedback Modified
User Suspended
```

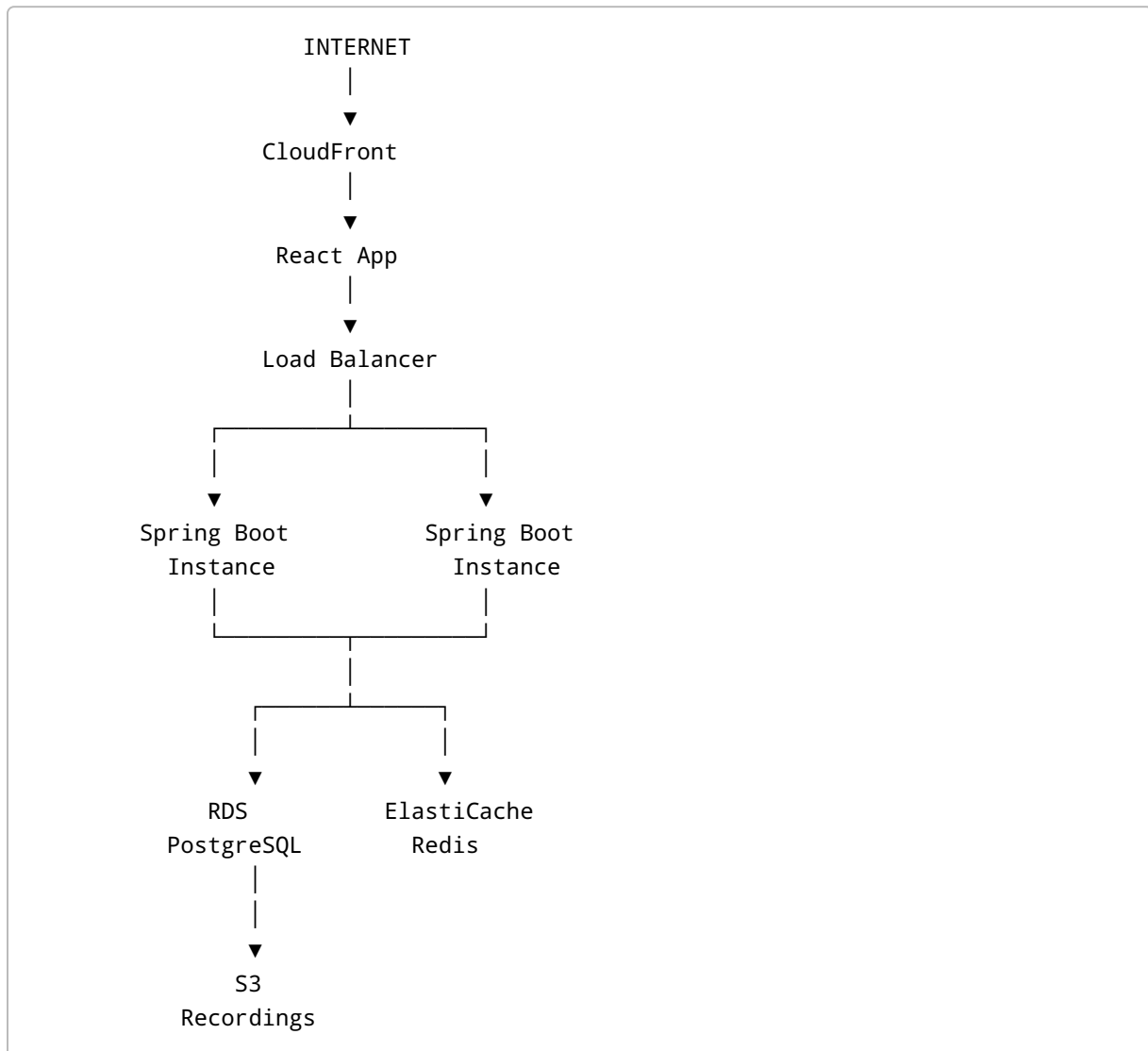
Audit record:

```
user_id
action
entity_type
entity_id
```

timestamp
IP
metadata

29. Deployment Architecture

Initial AWS architecture:



30. Security Architecture

```
HTTPS
  ↓
Load Balancer
  ↓
Spring Security
  ↓
JWT
  ↓
RBAC
  ↓
Controller
  ↓
Service
  ↓
Repository
```

Sensitive storage:

```
Database Encryption
S3 Private Bucket
Signed URLs
Secret Manager
Environment Variables
```

31. API Layer

Recommended API prefix:

```
/api/v1
```

Authentication:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

Candidate:

```
GET /api/v1/candidates/me
PUT /api/v1/candidates/me
GET /api/v1/candidates/me/interviews
GET /api/v1/candidates/me/performance
```

Interviewer:

```
POST /api/v1/interviewers/apply
GET /api/v1/interviewers/me
PUT /api/v1/interviewers/me
POST /api/v1/interviewers/availability
GET /api/v1/interviewers/me/interviews
```

Marketplace:

```
GET /api/v1/companies
GET /api/v1/domains
GET /api/v1/designations
GET /api/v1/interviewers/search
```

Booking:

```
POST /api/v1/bookings
GET /api/v1/bookings/{id}
POST /api/v1/bookings/{id}/cancel
```

Payment:

```
POST /api/v1/payments/order
POST /api/v1/payments/webhook
GET /api/v1/payments/{id}
```

Interview:

```
GET /api/v1/interviews/{id}
POST /api/v1/interviews/{id}/join
POST /api/v1/interviews/{id}/end
```

Feedback:

```
POST /api/v1/interviews/{id}/feedback
GET /api/v1/interviews/{id}/feedback
```

Recording:

```
GET /api/v1/interviews/{id}/recording
```

Admin:

```
GET /api/v1/admin/interviewers
POST /api/v1/admin/interviewers/{id}/approve
POST /api/v1/admin/interviewers/{id}/reject
GET /api/v1/admin/payments
GET /api/v1/admin/interviews
```

32. Architecture Principles

The system should follow:

```
SOLID
Clean Code
REST
DTO Pattern
Service Layer
Repository Pattern
Dependency Injection
Global Exception Handling
Validation
Logging
Audit Logging
RBAC
API Versioning
```

The system should avoid:

```
Business logic in controllers
Returning entities directly
```

```
Hardcoded pricing
Hardcoded domains
Hardcoded designations
Hardcoded company data
Public recording URLs
Frontend-only security
```

33. Future Microservice Architecture

If the platform grows significantly, modules can be extracted.

Potential future services:

```
API Gateway
|
├─ Auth Service
├─ User Service
├─ Interviewer Service
├─ Marketplace Service
├─ Scheduling Service
├─ Booking Service
├─ Payment Service
├─ Interview Service
├─ Video Service
├─ Recording Service
├─ Feedback Service
└─ Notification Service
```

However, this should be a future evolution rather than the initial implementation.

END OF ARCHITECTURE DOCUMENT